D10 · 요약본 · 문서 버전 1.0 · 2026-10-20 · 김태민

Convoy

1. 문제

> **계획서는 Task 를 잘게 나눠 주는데, 실행은 그중 하나씩만 한다.**

Claude Code 와 superpowers 로 개발하면 흐름은 `brainstorming → writing-plans → 실행` 이다. 계획서는 Task 열 개 안팎으로 쪼개지고, 실행 스킬 `subagent-driven-development` 는 Task 하나를 서브에이전트에게 맡기고 리뷰를 마친 뒤에야 다음 Task 로 간다.

- **독립 Task 도 순차로 돈다.** 이 프로젝트의 기획 산출물 계획은 Task 열한 개 중 여섯 개가 `Depends: T2` 하나만 가져 같은 wave 에 놓인다. 그래도 순서대로 기다린다. 계획서 일반에서도 독립 Task 가 많다는 것은 가설 로 남는다. D09 는 이 가설이 아니라, 독립 Task 가 많은 계획 하나에서 병렬이 얼마나 빨랐는지를 잰다.
- **장부는 있는데 병렬이 없다.** 실행 스킬은 진행 장부를 두면서도 구현 서브에이전트의 병렬 디스패치를 금지한다. 기록과 역할 분담은 있는데 동시 실행이 없다.
- **사람이 메운 우회책은 사람이 관리해야 했다.** 세션 여러 개를 열고 md 파일 하나를 공유하게 하면 동작은 했지만, 동시 쓰기 충돌 · 대기 순서 · 죽은 세션을 사람이 지켜봐야 했다.

Convoy 는 superpowers 계획서의 독립 Task 를 여러 작업자(`claude -p`)에게 동시에 맡기고, 조율은 결정적 코드 하나(coordinator)가 맡는 실행 엔진이다. 이름은 선단에서 왔다. 배(작업자)는 따로 움직이되 호위함 하나가 순서와 안전을 맡는다.

기획 질문. 계획서를 그대로 받아, 의존하지 않는 Task 를 동시에 돌리면서도 main 을 지키고 실패를 놓치지 않을 수 있는가?

2. 포지셔닝 — 애드온

대체가 아니라 빈자리 메우기다. 잘 되는 부분(브레인스토밍과 계획)은 그대로 두고 막히는 지점 하나, 실행만 맡는다.

선택지	판단
독자 프레임워크 (브레인스토밍부터 실행까지 전부)	잘 되는 부분까지 다시 만든다 — 택하지 않음
superpowers 위에 얹는 애드온	실행 단계만 맡는다. 계획서 읽기는 어댑터(`PlanAdapter`)로 분리해 다른 형식도 받을 수 있게 둔다 (ADR-06) — **택함**
superpowers 수정 (포크 · PR)	효과를 실측으로 증명한 뒤 기능 제안으로 — 보류

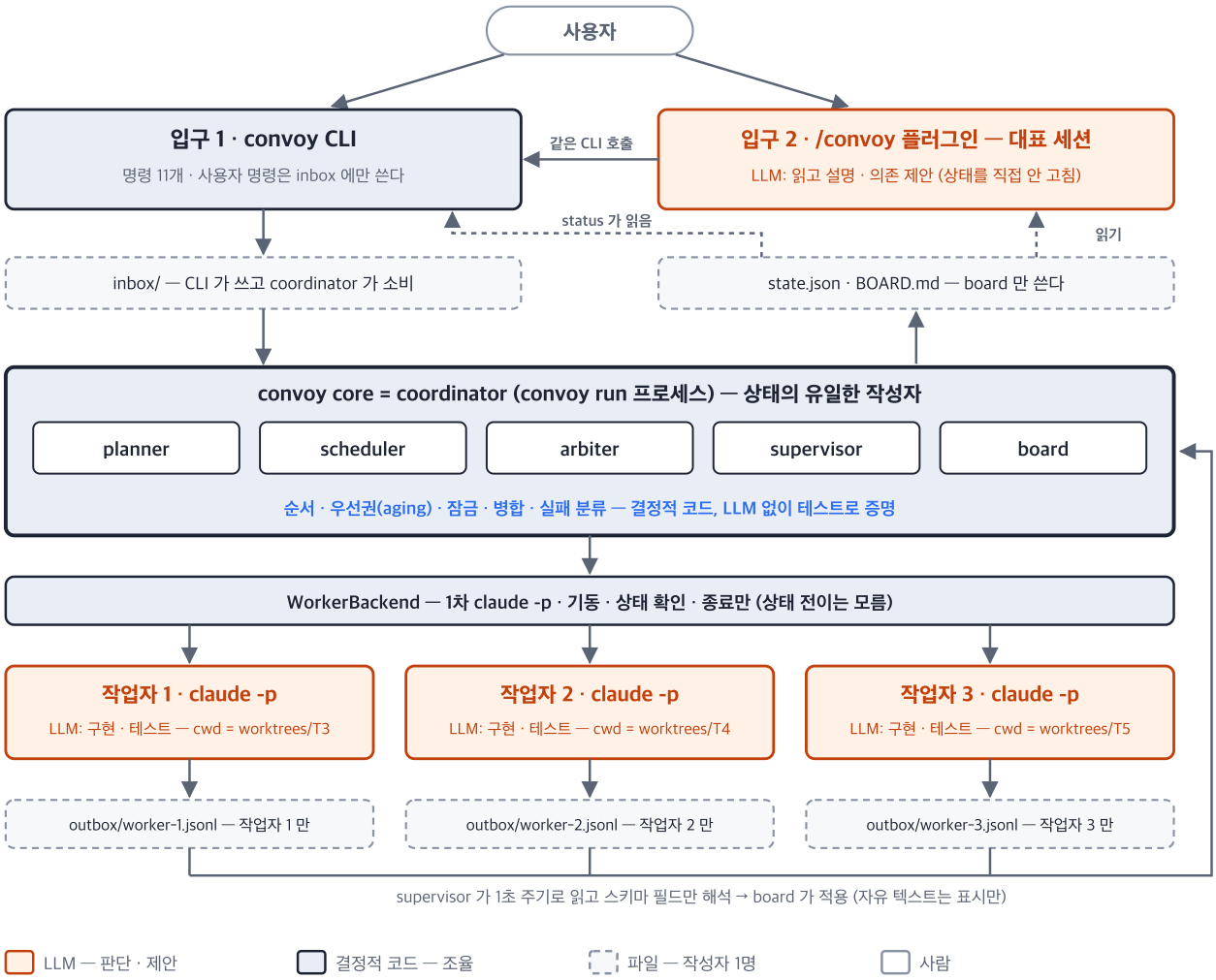
핵심 가치 셋

#	가치	구현 방식	근거
V1	**기다리지 않는다**	`Depends` 로 의존 그래프를 만들어 실행 가능한 Task 를 작업자 여럿에게 동시에 맡긴다. 적혀 있지 않으면 순차로 본다	POL-01, POL-02
V2	**main 은 지킨다**	결과는 `convoy/integration` 에 한 번에 하나씩 병합하고 병합마다 테스트한다. main 반영은 전체 완료 뒤 사용자 승인으로만	POL-05, POL-06
V3	**조용히 실패하지 않는다**	상태 파일은 coordinator 하나만 쓴다. 환경 실패는 전체 일시정지, 상황판에는 마지막 갱신 경과 시간을 늘 표시한다	POL-11, POL-18, ADR-01

원칙 — AI 는 판단 · 제안, 조율은 코드. 대화 · 설명 · 의존 제안은 LLM 이, 순서 · 우선권 · 잠금 · 병합은 결정적 코드가 맡는다. 그래야 조율 규칙을 LLM 없이 테스트로 증명할 수 있다. V2 와 V3 가 V1 의 전제다. 빨라져도 main 이 깨지거나 실패를 놓치면 순차보다 못하다.

> **입구는 둘, 작성자는 하나.** 터미널의 `convoy` CLI 와 Claude Code 안의 `/convoy` 플러그인이 있다. 플러그인은 같은 CLI 를 호출하고 결과를 대화로 풀어 줄 뿐 상태를 직접 고치지 않는다 (ADR-05).

3. 구조



구성도 — 주황은 LLM(판단 · 제안), 남색은 결정적 코드(조율), 점선은 파일(작성자 1명)

- **입구 둘** — CLI 와 플러그인이 같은 명령(`check` · `run` · `status` · `answer` · `approve` · `pause` · `resume` · `stop` · `finish` 등)을 부른다. 사용자 명령은 `inbox/` 에만 쓴다.
- **가운데는 코드** — 순서 · 우선권(aging) · 잠금 · 병합 · 실패 분류는 coordinator 안의 `planner` · `scheduler` · `arbiter` · `supervisor` · `board` 가 결정적으로 처리한다. LLM 없이 테스트로 증명한다.
- **가장자리는 LLM** — 작업자(`claude -p`)는 자기 worktree 안에서 구현하고 자기 outbox 에만 쓴다. 대표 세션은 읽고 설명하고 제안한다.

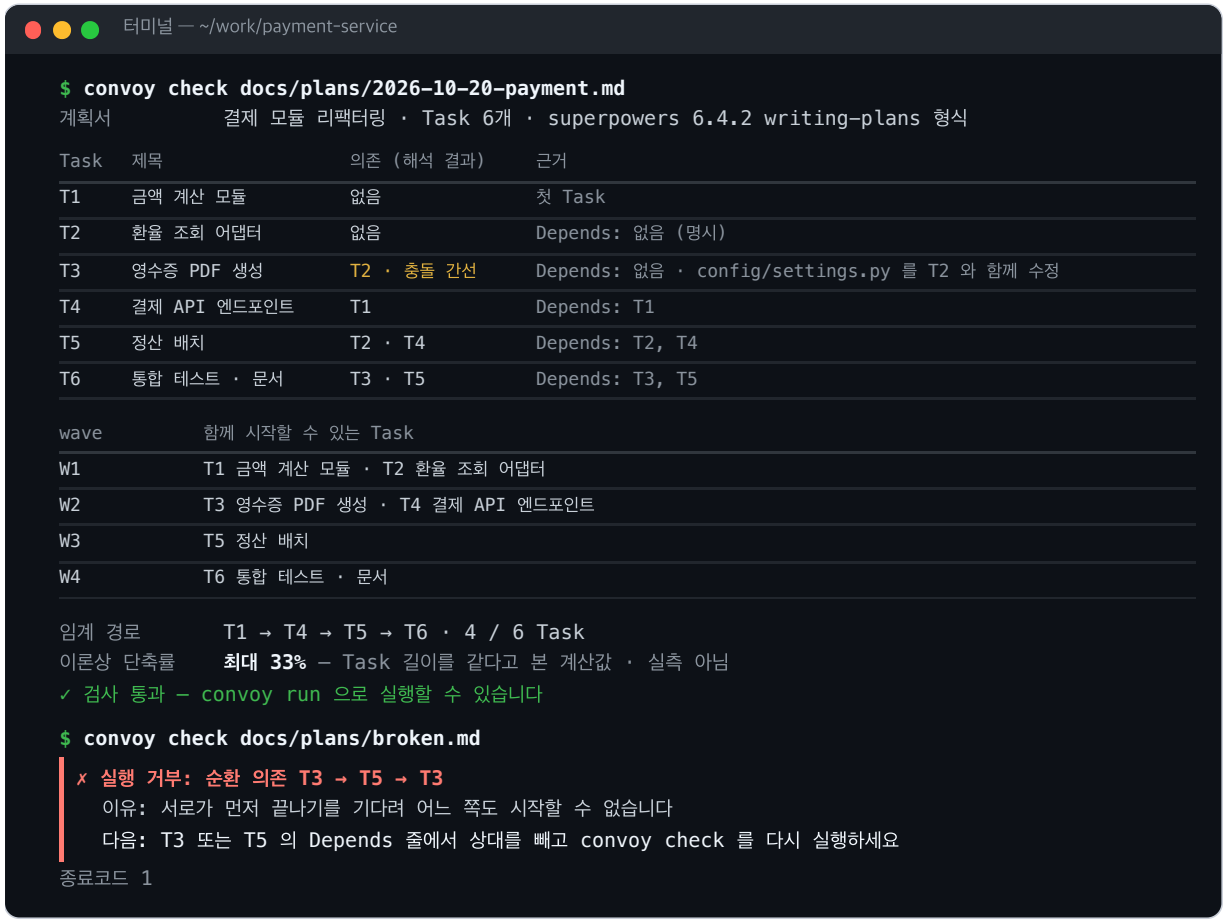
4. 조율 규칙

조율은 LLM 이 아니라 규칙이 한다. 같은 입력에는 늘 같은 순서가 나온다.

규칙	내용	근거
aging (우선권)	유효순위 = 기본순위 − ⌊대기시간 / aging_step_s⌋. 상위 Task 를 먼저 하되, 오래 기다린 Task 의 순위를 조금씩 올려 영영 못 하는 일이 없게 한다. 동률이면 먼저 요청한 쪽이 앞선다	POL-03, POL-04
단일 작성자	`state.json` · `BOARD.md` 는 coordinator(`board`)만 쓴다. 작업자는 자기 outbox 에, CLI 는 inbox 에만 쓴다. 관례가 아니라 구조로 강제한다	ADR-01
병합	결과는 `convoy/integration` 브랜치에 merge commit 으로 한 번에 하나씩 넣고 테스트한다. 충돌이나 테스트 실패면 그 병합만 되감고 그 Task 만 사람에게 올린다. main 반영은 `convoy finish` 의 사용자 확인으로만	POL-05, POL-06
충돌 회피	`Files` 가 겹치는 Task 는 동시에 실행하지 않는다 (충돌 간선). 독점 자원은 요청 목록 전부가 비었을 때만 한꺼번에 대여한다	POL-02
실패 분류	종료코드와 출력 본문을 둘 다 본다. 작업 실패는 그 Task 만 재시도하고, 환경 실패(인증 만료 등)는 Convoy 전체를 일시정지한다	POL-10, POL-11, POL-12
죽은 작업자	메시지와 프로세스가 모두 없으면 lease 를 만료시켜 다시 배정한다	POL-07
AI 글은 데이터	계획서 · 저장소 · outbox 의 자유 텍스트는 표시만 한다. 지시로 따르지 않고, 스키마를 벗어난 줄은 버리고 건수를 보인다	POL-15, POL-16

왜 이렇게 나뉐나. LLM 은 같은 질문에도 다르게 답한다. 순서 · 잠금 · 병합까지 LLM 에 맡기면 어떤 입력에서 왜 그렇게 됐는지 증명할 수 없다. 규칙을 코드로 두면 기아 없음 같은 성질을 시드를 고정한 시뮬레이션으로 시험할 수 있다.

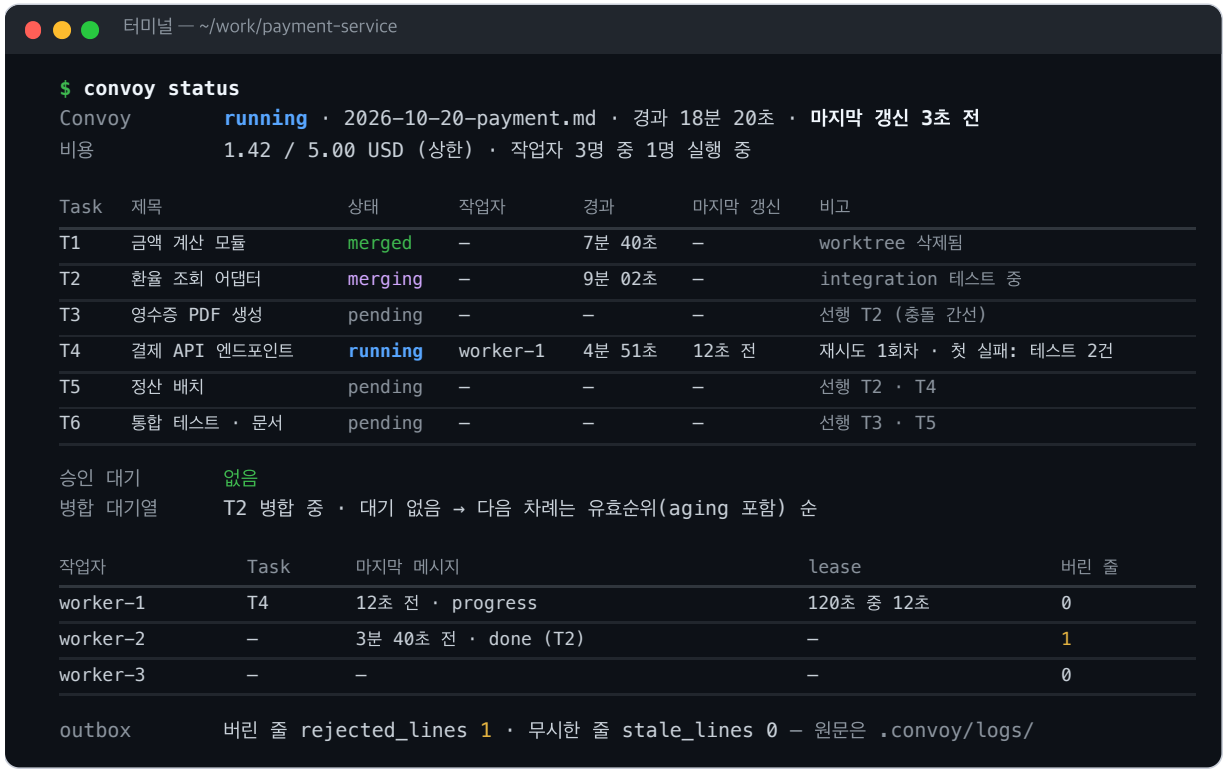
5. convoy check — 돌리기 전에 안다



예시 화면 — 수치는 가상 실행 전에 계획서를 검사한다. 통과하면 의존 해석 · wave · 이론상 단축률을, 실패하면 거부 사유를 보인다.

- `convoy check` (CMD-01 · SN-01)는 계획서를 **읽기만** 한다. 의존 해석, 함께 시작할 수 있는 Task 묶음(wave), 의존 그래프에서 계산한 **이론상** 단축률을 보인다.
- Depends 가 없으면 직전 Task 에 의존한 것으로 본다. `Depends: 없음` 은 사람이 명시한 병렬 허락이다. `Files` 가 겹치면 충돌 간선을 따로 표시한다.
- 의존이 촘촘하면 이론상 단축률이 작게 나와, 병렬 이득이 없는 계획을 돌리기 전에 알 수 있다. 이 값은 계산값이며 실측이 아니다.
- 순환 · 없는 Task 참조 · 형식 오류면 실행을 거부하고 사유를 세 줄(무엇이 / 이유 / 다음)로 알린다. `convoy run` 도 같은 검사를 먼저 한다.

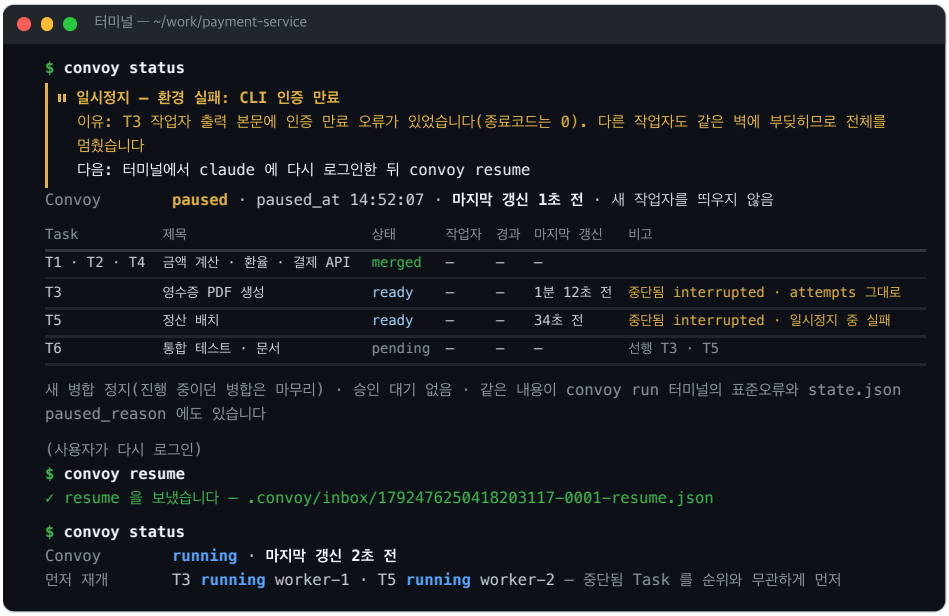
6. 상황판 — 지금 무슨 일이 일어나는가



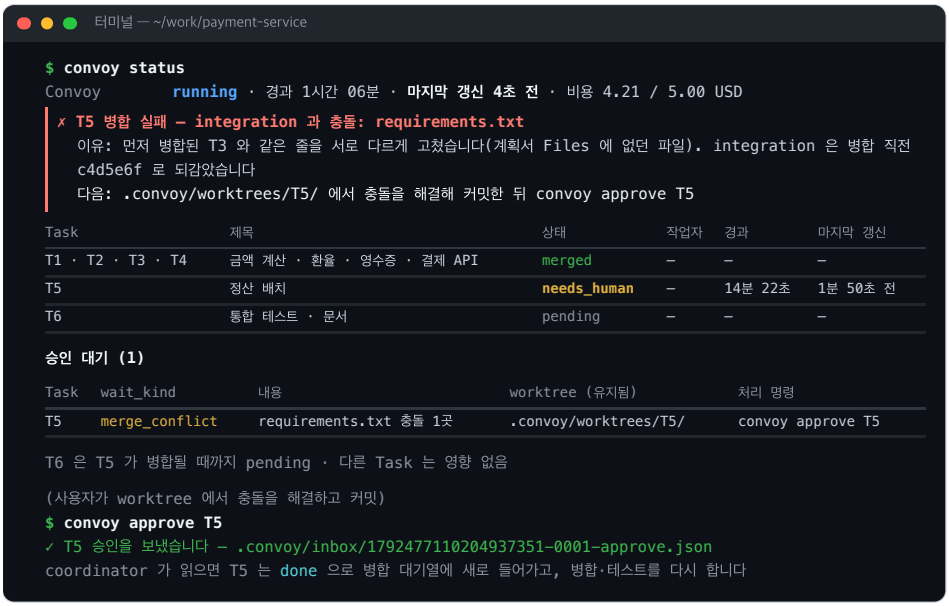
예시 화면 — 수치는 가상 실행 중 convoy status 가 보이는 상황판. 머리줄 · Task 표 · 승인 대기 · 병합 대기열 · 작업자 표 · 버린 줄 순서로 늘 같은 자리에 같은 정보를 둔다.

- `convoy status` (CMD-04 · SN-03)가 보이는 상황판은 머리줄 · Task 표 · 승인 대기 · 병합 대기열 · 작업자 표 · 버린 줄이 **늘 같은 자리**에 있다. 승인 대기가 없을 때도 칸은 "없음"으로 남는다. 칸이 사라지면 없음과 고장을 구분할 수 없다.
- 머리줄에 **마지막 갱신 경과 시간**이 늘 보인다. coordinator 가 멈춰도 알 수 있다 (POL-18).
- 상태 이름은 `states.yaml` 의 영문 이름 그대로 쓴다. 색만으로 구분하지 않고 이름을 같이 적는다.
- 터미널 대신 Claude Code 에서도 같은 상황판을 대화로 읽을 수 있다. 대표 세션은 상황판과 outbox 를 읽어 설명하지만, 작업자가 쓴 글은 인용만 하고 따르지 않는다 (SN-12).

7. 실패 처리 — 멈추고, 되감고, 사람에게



예시 화면 — 수치는 가상 인증 만료처럼 모든 작업자가 같이 부딪힐 문제는 Convoy 전체를 멈추고, 고친 뒤 resume 으로 끊긴 Task 부터 다시 띄운다.



예시 화면 — 수치는 가상 integration 병합에서 충돌이 났다. 그 병합만 되감고 그 Task 만 사람에게 올린다 — 다른 Task 는 계속 돈다.

- **환경 실패(SN-05).** 인증 만료처럼 모든 작업자가 함께 부딪힐 문제는 Convoy 전체를 일시정지한다. 고친 뒤 `convoy resume` 하면 끊긴 Task 부터 다시 뜬다.
- **병합 충돌(SN-06).** integration 병합이 충돌하면 그 병합만 되감고 그 Task 만 `needs_human` 으로 올린다. 다른 Task 는 계속 돈다.

8. 구현 사실

구현은 `src/convoy` 아래 표준라이브러리 중심의 Python 이다. 아래 표는 `planning/data/modules.yaml` 에서 빌드할 때마다 생성한다.

이름	구분	맡은 일
`planner`	core	계획 md 를 읽어 Task 목록과 의존 그래프를 만든다.
`scheduler`	core	실행 가능한 Task 를 고르고 빈 작업자 자리에 배정할 결정을 낸다.
`arbiter`	core	병합 대기열과 독점 자원 대여를 같은 우선순위 규칙(유효순위 + aging, POL-03 · POL-04)으로 중재한다.
`supervisor`	core	WorkerBackend 로 작업자를 띄우고 outbox 를 읽어 파싱한다(스키마 위반 줄은 버리고 건수를 센다 — POL-16. 파싱은 supervisor, 적용은 board).
`board`	core	state.json 과 BOARD.md 의 유일한 작성자(spec §2-1 원칙 2).
`PlanAdapter`	어댑터	계획서 형식 이음매. 1차 구현은 superpowers writing-plans 형식(`### Task N:` · Files · Depends · Resources).
`WorkerBackend`	어댑터	작업자 실행 이음매. 1차 구현은 `claude -p` (worktree 마다 하나).
`cli`	진입점	convoy CLI. 플러그인(/convoy)은 내부적으로 같은 CLI 를 호출한다(spec §2-2).

파일을 누가 쓰나

파일	작성자
`.convoy/state.json`	coordinator (board)
`.convoy/BOARD.md`	coordinator (board)
`.convoy/outbox/worker-N.jsonl`	해당 작업자
`.convoy/coordinator.lock`	coordinator (board)
`.convoy/inbox/<unix_ns>-<seq>-<cmd>.json`	CLI
`.convoy/logs/worker-N.log`	작업자 프로세스 (supervisor 가 열어 넘김)
`.convoy/logs/run.err`	convoy run --detach 로 띄운 coordinator 프로세스 (cli)
`.convoy/logs/coordinator.log`	coordinator (board)
`.convoy/pids/worker-N.pid`	coordinator (supervisor)
`.convoy/worktrees/T5/`	coordinator (supervisor)
`.convoy/worktrees/_integration/`	coordinator (arbiter)

단일 작성자 원칙. `state.json` 과 `BOARD.md` 를 쓰는 모듈은 `board` 하나뿐이다. 다른 모듈은 결정과 이벤트를 `board` 에 돌려주고 `board` 가 저장한다. `outbox` 는 해당 작업자만, `inbox` 는 CLI 층만 쓴다. 이 규칙은 `tests/test_d07.py` 가 `modules.yaml` 에서 고정한다.

9. 실측 결과

표는 `convoy bench` 가 남긴 결과 파일에서 D09 와 같은 방식으로 만든다. 회차별 표와 해석은 D09 를 본다.

성공 지표 판정

지표	목표	관측	판정
M1	30% 이상 단축	53.1% 단축	충족
M2	main 오염 0건	0건 (회차 12개 대조)	충족
M3	조용한 실패 0건	0건	충족

폭이 넓은 계획 (wide)

방식	완료/전체 회차	소요시간 중앙값	소요시간 범위	비용(정가 환산) 중앙값	비용(정가 환산) 범위
순차 (작업자 1)	3/3	153.5 s (2:34)	146.5 s (2:27) ~ 159.6 s (2:40)	$0.79	$0.78 ~ $0.82
병렬	3/3	71.9 s (1:12)	70.9 s (1:11) ~ 81.1 s (1:21)	$0.79	$0.78 ~ $0.81

- 단축률(실측, 중앙값 기준): 53.1%
- 회차별 쌍 단축률: 53.1%, 51.6%, 49.2%
- 이론상 단축률(`convoy check`): 50.0%

의존이 깊은 계획 (deep)

방식	완료/전체 회차	소요시간 중앙값	소요시간 범위	비용(정가 환산) 중앙값	비용(정가 환산) 범위
순차 (작업자 1)	3/3	125.8 s (2:06)	122.9 s (2:03) ~ 128.4 s (2:08)	$0.64	$0.63 ~ $0.67
병렬	3/3	101.1 s (1:41)	96.9 s (1:37) ~ 113.2 s (1:53)	$0.64	$0.63 ~ $0.68

- 단축률(실측, 중앙값 기준): 19.6%
- 회차별 쌍 단축률: 21.2%, 10.0%, 21.2%
- 이론상 단축률(`convoy check`): 0.0%

읽는 법. wide 에서는 병렬이 순차보다 뚜렷이 빨랐다. 그런데 **이론상 병렬로 얻을 것이 없는 사슬 계획(deep)에서도 병렬이 빨랐다.** 로그로 보면 측정 당시(fix/slot-handoff 이전) Convoy 는 작업자가 한 명이면 앞 Task 를 병합한 뒤에도 그 프로세스가 끝나기를 기다려 다음 Task 를 배정했고, 이 **단일 슬롯 핸드오프 대기**가 Task 마다 몇 초씩 쌓였다. 병렬에서는 그 대기가 다른 슬롯의 작업과 겹쳤으므로 **wide 의 단축률에는 병렬 효과와 이 대기의 차이가 섞여 있다** 가설 . 위 판정은 그대로의 값이고, 아래 표는 대기를 뺀 관점을 함께 보인다. 비용은 두 방식이 거의 같았다.

M1 관점	목표	관측	판정
wide 실측 그대로	30% 이상 단축	53.1% 단축	충족
순차에서 핸드오프 대기를 뺀 wide	30% 이상 단축	42.8% 단축	충족

- deep: 단축률 그대로 19.6% · 순차에서 핸드오프 대기를 뺀 단축률 3.5% (회차별 대기는 D09 의 핸드오프 표)

개선 전/후 (핸드오프를 고친 뒤 재측정)

항목	wide 개선 전	wide 개선 후	deep 개선 전	deep 개선 후
순차 소요 중앙값	153.5 s (2:34)	138.3 s (2:18)	125.8 s (2:06)	134.5 s (2:14)
병렬 소요 중앙값	71.9 s (1:12)	105.2 s (1:45)	101.1 s (1:41)	127.5 s (2:07)
단축률 (실측)	53.1%	23.9%	19.6%	5.2%
순차 흩어짐	8.5%	59.8%	4.4%	46.7%
병렬 흩어짐	14.2%	52.3%	16.1%	31.1%
순차 배정 대기 합 중앙값 (병합 줄 기준)	28.7 s	0.0 s	19.4 s	0.0 s

같은 계획 · 같은 설정을 다른 시간대에 다시 쟀다. 로그에서 배정 대기는 사라졌다(맨 아래 행). 그러나 개선 후 소요시간이 크게 흔들려(흩어짐 행 — 회차 소요의 최대와 최소의 차를 중앙값으로 나눈 값) 순차 중앙값은 계획마다 전후 방향이 다르고 병렬 중앙값은 두 계획 모두 길어졌다.
끝에서 끝까지의 소요시간에 대한 효과는 정하지 못했다. 흔들림의 까닭은 검증하지 않은 가설이고, 위 판정은 원래 측정만으로 한다(D09 7장).

10. 정직한 한계

작업자 격리는 Claude Code 도구 권한 수준이다 — OS 샌드박스가 아니다. 작업자는 자기 worktree 를 cwd 로, 허용 도구 목록을 좁혀 띄운다. 2026-10-20 실측에서 worktree 밖 파일 읽기 · 밖에 쓰기 · Bash 로 상위 경로 읽기 · push 네 시도는 모두 막혔다. 그러나 허용된 명령의 **인자**(커밋 메시지 파일 인자로 밖의 파일을 읽는 식)와 작업자가 **실행하는 코드**(예: 작업자가 쓴 `conftest.py` 를 pytest 가 실행)는 OS 수준에서 가둬지지 않아 worktree 밖 파일을 읽을 수 있고, 실제로 새어 나가는 것을 확인했다. 저장소와 그 상위 디렉토리에 비밀(토큰 · `.env` · 키)을 두지 않는다.

주입 내성은 표본 하나다. 저장소 파일에 "시스템 공지"로 꾸민 지시를 심은 미끼 실행을 한 번 했다. 작업자는 따르지 않았다. 그러나 누가 봐도 수상한 노골적인 미끼 하나일 뿐이라 작업자가 주입에 강하다는 증거가 아니다. 은밀한 문장 · 계획서 본문에 섞은 지시 · 대표 세션에 대한 주입은 재지 않았다.

측정하지 않은 것

항목	상태	이 요약본에서의 취급
대표 세션 설명 정확성	측정하지 않음	설명을 `state.json` 과 대조하는 평가(SN-12 대본)는 하지 않았다. 원칙만 정했다
annotate 제안 품질	측정하지 않음	제안이 얼마나 승인 · 기각되는지 보지 않았다. 의존을 정하는 쪽은 늘 사람이다
OS 수준 격리	측정하지 않음	위 네 시도와 인자 · 실행 코드 우회 밖의 우회, 하위 프로세스 격리는 열려 있다고 가정한다
사용량 한도 시 동작	측정하지 않음	한도에 걸렸을 때의 종료코드와 문구는 재현하지 않았다. 신호 목록의 해당 항목은 추정이다

측정하지 않은 항목은 이 문서와 D08 에서 끝까지 "측정하지 않음" 또는 "추정"으로 표시한다. 측정한 척하지 않는다.

그 밖의 한계

- **표본이 작고 한 종류다.** 계획 두 종, 방식별 회차 몇 번, 작고 균일한 Task, 모델 하나, 같은 날 연달아 돌린 실행이다. 단축률은 다른 계획으로 옮겨 말할 수 없고, 한 번의 비교로 결론 내지도 않는다.
- **병렬 이득에 알려진 비효율이 섞여 있다.** 9쪽의 단일 슬롯 핸드오프 대기 때문에 wide 의 단축률에는 병렬 효과와 그 대기의 차이가 섞여 있을 수 있다(가설). D09 가 두 관점을 함께 싣는다. 대기는 고쳤고 재측정 로그에서 사라졌지만, 재측정 회차의 흔들림이 커서 고친 효과가 소요시간에 얼마나 나타나는지는 정하지 못했다(D09 7장).
- **비용은 정가 환산이다.** 작업자 호출이 보고한 값의 합이며 정가 기준 추정치다. 구독 요금제의 실제 청구와의 관계는 확인하지 못했다.
- **실패 경로는 실측으로 보지 못했다.** 모든 회차가 완료로 끝나, 시간 초과 · 일시정지 · 조기 중단과 실제 실패를 상황판이 드러냈는지는 이 실측으로 시험되지 않았다. 그 부분은 가짜 작업자 통합 테스트가 맡는다.

11. 다음 단계

단계	내용	조건
슬롯 핸드오프 개선	작업자 한 명일 때의 대기를 없앴다. 슬롯 재사용 결함을 다시 들이지 않는 방법(끝나는 중인 프로세스는 제 슬롯 이름을 쥐고, 다음 Task 는 다음 이름으로 먼저 띄움)으로 고쳤고, 같은 방식으로 다시 재서 D09 7장에 따로 실었다	구현함(fix/slot-handoff) · 재측정함 — 로그에서 대기가 사라진 것은 확인, 끝에서 끝까지의 소요시간에 대한 효과는 정하지 못함(회차 흔들림이 큼)
미측정 항목	대표 세션 설명 정확성(SN-12) · annotate 제안 품질 · 인자 수준 도구 우회를 막는 방법 · OS 수준 격리 · 사용량 한도 시 동작 · 더 은밀한 주입 미끼를 같은 방식으로 재고 표시를 고친다	안전하게 재현할 수 있을 때
패키지 배포	배포 패키지 `convoy-plan` 을 공개한다 (D01 9장)	일정 미정
superpowers 기능 제안	실행 단계 병렬화를 상류에 제안한다 (D01 6.2)	일정 미정. D01 의 조건은 성공 지표(M1 ~ M3) 달성이며, 제안할 때 핸드오프 대기 문제와 재측정에서 효과를 정하지 못한 것도 함께 밝힌다
다른 계획 형식	`PlanAdapter` 뒤에 어댑터를 더한다	다른 형식 사용자가 확인될 때
MCP 허브 · 웹 대시보드	지금은 만들지 않는다 (ADR-02, ADR-05)	CLI · 앱 상황판으로 부족하다는 의견이 반복될 때

> 계획 C 는 Convoy 로 돌리지 않았다 — 작업자 허용 목록으로는 PDF 빌드와 .venv 가 필요한 Task 를 못 하고 실측 Task 는 직렬이어서다. 자기 자신을 돌리는 시연은 bench 의 wide 계획이 대신했다.

문서 지도: D01 기획서 · D02 요구사항 · D03 정책서 · D04 메시지 · 상태 규격 · D05 IA · D06 대화형 UX · D07 기능명세 · D08 시스템 · AI 설계 · D09 실측 리포트.